

Group 11 Requirements Analysis

Fully Dressed Use Cases:

Ally: UC-A1: Report Lost Item

Primary actor: User (Lost Item Owner)

Stakeholders and interests: User: Wants a fast, simple way to report a lost item so others can find it. Campus Community: Needs accurate information to help return items. System: Needs structured data for search, matching, and notifications.

Preconditions: User is logged into the app, User has an active account.

Success guarantee: The lost item is successfully posted and visible in the system.

Main success scenario:

1. User selects “Report Lost Item.” System displays the lost item form.
2. User enters item details (title, description, date lost, location lost).
3. User selects a category (e.g., electronics, clothing).
4. User uploads optional photos.
5. User submits the form.
6. System validates the information.
7. System creates a new LostItem record.
8. System displays confirmation and shows the item in “My Items.”

Extensions: 3a. Required fields missing → System prompts user to complete them.

5a. Photo upload fails → System allows retry or skip.

7a. Validation fails → System highlights errors.

No special requirements.

Technology and data variations: User may upload 0–5 photos depending on device capability.

UC-A2: Update Lost Item Status

Primary actor: User (Lost Item Owner)

Stakeholders and interests: User: Wants to update the item’s status so others know it is no longer missing. Claimants: Need accurate status to avoid confusion or wasted effort. System: Must maintain synchronized, real-time status across all devices.

Preconditions: User is logged into the app, User has at least one active lost item posted.

Success guarantee: The item’s status is updated and visible system-wide in real time.

Main success scenario:

1. User opens “My Items.”
2. User selects a lost item.
3. User chooses “Update Status.”
4. System displays available status options (Active, Found).
5. User selects a new status.
6. System updates the LostItem record.
7. System notifies any pending claimants.
8. Updated status appears immediately in the app.

Extensions: 5a. User cancels → No changes saved. 6a. Network error → System retries or prompts user to try again.

No special requirements.

Technology and data variations: Status update may trigger push notifications or in-app alerts.

UC-A3: View & Track Claim Requests for My Lost Item

Primary actor: User (Lost Item Owner)

Stakeholders and interests: User: Wants to review claim requests and verify legitimate claimants. Claimants: Want timely responses to their claim submissions. System: Must maintain accurate claim histories and associations.

Preconditions: User is logged into the app, User has at least one lost item with claim requests.

Success guarantee: User can view all claim requests and their statuses.

Main success scenario:

1. User opens “My Items.”
2. User selects a lost item.
3. User taps “View Claim Requests.”
4. System retrieves all ClaimRequest records.
5. System displays each request with claimant info, timestamp, and status.
6. User reviews claim details.
7. User may contact claimant or verify ownership.

Extensions: 4a. No claim requests exist → System displays “No claims yet.” 5a. Some claimant info unavailable → System displays partial data.

No special requirements.

Technology and data variations: Claim details may include photos or proof-of-ownership messages.

UC-A4: Tag Lost Item with Category

Primary actor: User (Lost Item Owner)

Stakeholders and interests: User: Wants accurate categorization to improve search visibility. Search System: Needs consistent categories for filtering and matching. Campus Community: Benefits from organized listings.

Preconditions: User is logged into the app, User is reporting or editing a lost item.

Success guarantee: The item is assigned a valid category.

Main success scenario:

1. User opens the lost item form (new or edit).
2. System displays a list of predefined categories.
3. User selects a category (e.g., electronics, clothing).
4. System validates the selection.
5. Category is saved to the LostItem record.
6. System updates search and filtering indexes.

Extensions: 2a. Category list fails to load → System retries or shows cached categories. 3a. User selects “Other” → System prompts for a custom category.

No special requirements.

Technology and data variations: Category list may be updated by Admins over time.

Malavika: UC-M1: Report Found Item

Primary actor: User (Finder)

Stakeholders and interests:

- User: Wants a quick way to report an item they found so it can be returned
- Potential Owner: Wants the accurate details to find their item
- Campus Community: Benefits from more items being returned instead of staying lost
- System: Needs structured item data for search, matching, and claim workflows

Preconditions: The user is logged into the app, and the user has an active account.

Success guarantee: A found item post is created and visible in the system with status ‘Available.’

Main success scenario:

1. User selects “Report Found Item.”
2. System displays the found item form.
3. User enters item details (title, description, date found, location found).
4. User selects a category and optionally adds photos.
5. User submits the report.
6. System validates required fields.
7. System creates a new FoundItem record with status Available.
8. System displays confirmation and shows the post under “My Found Items.”

Extensions:

- 3.a Missing required fields -> System highlights missing fields and prompts user to complete them
- 4a. Photo upload fails -> System allows retry or submit without photos.
- 6a. Validation fails -> System shows an error message and blocks submission.
- 7a. Network/server error -> System saves as draft (if supported) or asks user to retry.

No special requirements.

Technology and data variations: The user may upload 0–5 photos depending on device capability. Location may be entered manually or selected from a map/preset campus locations.

UC-M2: Update Found Item Status

Primary actor: User (Finder)

Stakeholders and interests:

- User: Needs to keep the post accurate (available vs. claimed).
- Claimants: Need accurate status so they don't submit claims for already-claimed items.
- System: Must keep status consistent across search results and claim workflows.

Preconditions: User is logged into the app. User has at least one found item post they created.

Success guarantee: The found item's status is updated and reflected system-wide.

Main success scenario:

1. User opens "My Found Items."
2. User selects a specific found item post.
3. User chooses "Update Status." System displays status options (e.g., Available, Claimed).
4. User selects a new status.
5. System updates the FoundItem record.
6. System updates listings/search results to match the new status.
7. System shows confirmation.

Extensions:

- 5a. User cancels → No changes are saved.

- 6a. User attempts to set status to Claimed while a proof step is required → System redirects to the proof verification flow.
- 6b. Network error → System retries or prompts user to try again.

No special requirements.

Technology and data variations: Status updates may trigger push notifications or in-app alerts to interested users (if enabled).

UC-M3: Accept or Decline Claim Requests

Primary actor: User (Finder).

Stakeholders and interests:

- User: Wants control to approve legitimate claims and reject suspicious ones.
- Claimant: Wants timely decisions and clear outcomes.
- System: Must record decisions and maintain an auditable claim history.

Preconditions: User is logged into the app. User has a found item post with at least one pending claim request.

Success guarantee: A claim request is marked Accepted or Declined, and the claimant is notified.

Main success scenario:

1. User opens “My Found Items.”
2. User selects a found item.
3. User taps “View Claim Requests.”
4. System displays all pending claim requests with claimant details and submitted info.
5. User opens a specific claim request to review details.
6. User selects Accept or Decline.
7. System records the decision on the ClaimRequest.
8. The system notifies the claimant of the decision.

Extensions:

- 3a. No claim requests exist → System displays “No claims yet.”
- 6a. User selects Accept → System requires proof-of-ownership verification before completion.
- 6b. User declines → System optionally requires a reason (if your app supports it), otherwise just declines.
- 8a. Notification fails → System still saves the decision and queues the notification for retry.

Special Requirements: If the claim is accepted, proof-of-ownership must be completed before the item can be finalized as claimed.

Technology and data variations: Claim requests may include text descriptions, photos, or identifying details (serial number, distinguishing marks).

UC-M4: Verify Proof of Ownership Before Making as Claimed

Primary actor: User (Finder)

Stakeholders and interests:

- Finder: Wants confidence that the item goes to the real owner.
- Claimant: Wants a fair way to prove ownership and retrieve their item.
- System/Campus Community: Needs fraud prevention to maintain trust in the platform.

Preconditions: User is logged into the app. A claim request has been selected for potential acceptance. The found item status is currently Available.

Success guarantee: Proof is recorded as verified, the claim is finalized as Accepted, and the found item can be marked Claimed.

Main success scenario:

1. User opens a pending claim request for a found item.
2. User selects Accept Claim.
3. System prompts for proof-of-ownership verification requirements (e.g., matching description, unique identifiers, photo evidence).
4. Claimant submits proof (text details and/or uploads).
5. System displays the submitted proof to the finder for review.
6. Finder confirms "Proof Verified."
7. System updates the claim request status to Accepted/Verified.
8. System sets the found item status to Claimed and closes other pending claims (or marks them as not selected).
9. System notifies the claimant with the next steps (pickup/meeting instructions if supported).

Extensions:

- 4a. Claimant does not provide required proof → System marks request as Incomplete and prompts claimant to resubmit.
- 6a. Finder chooses Proof Not Verified → System keeps the claim as Pending or sets it to Declined (based on your rules).
- 8a. Another claim already finalized first → System blocks the update and shows "Item already claimed."
- 9a. Notification fails → System logs the outcome and retries notification.

No special requirements.

Technology and data variations: Proof may be entered as free-text, selected from a checklist, or uploaded as images. Some items may require stronger proof (like electronics with serial numbers) vs. weaker proof (like clothing with descriptions).

Unnati: UC-U1: Advanced Search And Filtering

Primary actor: User (student/faculty/staff)

Stakeholders and interests:

- User: wants to quickly narrow down results and find a likely match.
- Campus community: wants fewer duplicate posts and faster item recovery.
- System owner/admin: wants search to be reliable and not time out.

Preconditions:

- User has access to the app (logged in or guest, depending on your team decision).
- There are item posts (lost/found) stored in the system.

Success guarantee:

- System displays a list of item posts that match the query + filters (or clearly shows “no results”).
- User’s chosen filters/sort are applied consistently.

Main success scenario: User opens the Search screen. User enters a keyword (ex: “AirPods”) or leaves it blank to browse. User selects one or more filters (category, item type lost/found, location, date range, status). User taps Search/Apply. System validates the filter combination and runs the search. System displays matching results sorted by relevance (or the selected sort). User taps a result to view item details.

Extensions:

3a. User selects conflicting filters (ex: “status=claimed” + “status=available” if those are mutually exclusive)

1. System highlights the conflict and asks user to revise filters.

5a. Search returns zero results

1. System shows “No matches found” and suggests removing filters or broadening the query.

5b. Search service temporarily unavailable

1. System shows an error and provides “Try again” (no data is changed).

Special Requirements: Search results should load fast enough to feel responsive.

Technology and data variations:

- Keyword can be typed or selected from recent searches.
- Filters can be applied via dropdowns, chips, sliders, or checkboxes.
- Location can be chosen from a campus list or set as “near me” (if allowed).

UC-U2: Item Matching Recommendations

Primary actor: User

Stakeholders and interests:

- Poster: wants help finding likely matches without manually searching forever.
- Other users: want accurate suggestions so they aren't spammed.

Preconditions:

- User has created or is viewing an item post (lost or found).
- The system has enough data (tags, category, text, location/time) to attempt matching.

Success guarantee:

- System displays a ranked list of recommended matching posts (or "no recommendations yet").
- No posts are modified automatically.

Main success scenario:

1. User opens a specific item post (their lost/found post).
2. User taps Suggested Matches (or it auto-shows).
3. System gathers key attributes from the post (category, tags/keywords, location, time range, color/brand if present).
4. System runs the matching process against opposite-type posts (lost ↔ found).
5. System shows recommended matches with a short "why this matched" hint (ex: same location + similar tags).

Extensions:

3a. Post has very little info

1. System suggests the user add missing details (category, location, date) to improve matching.

4a. Matching service cannot compute right now

1. System shows a retry option and does not lose the user's post.

Special Requirements: Recommendations should be understandable (basic "reason" text), not a black box.

Technology and data variations: Matching based on: tags, text similarity, location proximity, date/time proximity, optional image tag suggestions.

UC-U3: Notifications For Matched Items

Primary actor: User

Stakeholders and interests:

- User: wants to be alerted quickly when a potential match appears.

- System owner: wants notifications to be timely but not overwhelming.

Preconditions:

- User has at least one active lost/found post.
- User has notifications enabled (or default settings exist).

Success guarantee:

- User receives a notification for a new potential match and can navigate to the match.
- System logs that a notification was sent (for debugging / user settings).

Main success scenario:

1. A new item post is created (lost or found).
2. System checks whether it potentially matches any existing opposite-type posts.
3. System identifies one or more potential matches for the user's post.
4. System sends a notification to the affected user(s) with a short preview.
5. User taps the notification.
6. System opens the app to the matched item details page.

Extensions:

4a. User has notifications turned off

1. System does not send the alert (optional: show it in an in-app "Notifications" list).

4b. Too many matches in a short time

1. System batches notifications (ex: "3 new possible matches") instead of spamming.

Notification delivery and match-checking should remain scalable under heavy usage.

Technology and data variations:

- Push notification vs in-app notification only.
- User can configure match sensitivity (strict vs broad).

UC-U4: In-App Messaging Between Users

Primary actor: User

Stakeholders and interests:

- Users: want a safe, quick way to coordinate pickup/verification.
- Admin/team: wants messaging to be safe and reportable.

Preconditions:

- User is logged in.

- User is viewing an item post that allows contact (or a match screen).

Success guarantee:

- Message is delivered (or queued) and visible in both users' chat thread.
- Conversation is linked to the relevant item post.

Main success scenario:

1. User views an item post (or a suggested match).
2. User taps Message.
3. System opens (or creates) a conversation thread tied to that item.
4. User types a message (ex: where/when they found it, questions).
5. User sends the message.
6. System delivers the message and shows it in the thread for both users.

Extensions:

2a. Other user has disabled messages

1. System shows "Messaging unavailable" and suggests using claim workflow instead (if your team supports it).

6a. Delivery fails temporarily

1. System marks message as "Not sent" and allows retry.

5a. User reports inappropriate message

1. System records the report for admin review (ties into admin/reporting features).

Basic safety: allow block/report in chat (even if moderation is handled elsewhere).

Technology and data variations:

- Real-time messaging (WebSocket) vs near-real-time (polling).
- Attachments allowed or text-only (team decision).

Taeoh: UC - T1: Access Admin Dashboard & System Analytics

Primary actor: Admin

Stakeholders and interests:

- Admin: Wants to see high-level metrics of system usage and health.
- System: Needs to aggregate data into readable charts and tables.

Preconditions: User is logged in with Admin privileges.

Success guarantee: Admin sees a real-time overview of active items, successful returns, and user growth.

Main success scenario:

- Admin selects "Admin Panel" from the menu.
- System verifies administrative access control.
- System retrieves system-wide analytics (e.g., total items found this week, average time to claim).
- System displays data through an interactive dashboard.

Extensions:

- 2a. Unauthorized user attempt → System blocks access and logs the event.

Special Requirements: Dashboard must remain reliable even during high traffic

UC - T2: Moderate Item Content

Primary actor: Admin

Stakeholders and interests:

- Campus Community: Needs the platform to remain free of spam or inappropriate content.

Preconditions: Admin is logged into the Admin Dashboard.

Success guarantee: Inappropriate content is removed or edited.

Main success scenario:

1. Admin views the list of all active item posts.
2. Admin identifies a post that violates community guidelines (e.g., spam, illegal items).
3. Admin selects "Delete Post" or "Hide Post."
4. System updates the item record status to "Removed."
5. System logs the admin action for auditing purposes.

Extensions:

- 4a. Admin wants to warn the user → System provides a template to send a warning message.

UC-T3: Manage User Behavior Reports

Primary actor: Admin

Stakeholders and interests:

- Users: Want a safe environment where harassment or fraudulent claims are addressed.

Preconditions: A user has flagged another user or a message for inappropriate behavior.

Success guarantee: The report is reviewed and an appropriate administrative action is taken.

Main success scenario:

1. Admin opens the "User Reports" queue.
2. Admin reviews the reported content and the involved users' history.
3. Admin decides on an action: Dismiss, Warn User, or Suspend Account.
4. System applies the decision to the target User's account.
5. System notifies the reporting user that the issue has been resolved.

Extensions:

3a. Insufficient evidence → Admin requests more information or closes the report as "Dismissed."

Special Requirements: Maintain an auditable history of all disciplinary actions

UC-T4: System Performance Monitoring

Primary actor: Admin

Stakeholders and interests:

- Dev Team: Needs to know if the server is failing or if synchronization is lagging.

Preconditions: System is running.

Success guarantee: System logs and performance metrics are available for review.

Main success scenario:

1. Admin navigates to the "System Status" section of the dashboard.
2. System displays server uptime, database response times, and synchronization status
3. Admin identifies a performance bottleneck or a synchronization error.
4. Admin initiates a system diagnostic or notifies the technical team.
5. Technology and data variations: Metrics may include the number of active WebSockets for messaging and real-time updates

Dongjin: UC-T1: User registration

Primary Actor: Student, faculty, or staff

Stakeholders and interests:

- User: Wants a quick, straightforward way to create an account using university credentials.
- System Admin: Needs to ensure only authorized campus members register to maintain community trust.

Preconditions: User must have access to a valid university email address.

Success guarantee: A new user profile is created in the database and the user is granted access to the app's posting features.

Main success scenario:

1. User selects the "Sign Up" option on the landing page.
2. User enters their university email, a secure password, and their full name.
3. The system validates that the email follows the university format and is not already registered.
4. The system sends a verification link to the provided email address.
5. User clicks the link to verify the account.
6. The system activates the account and redirects the user to the Profile Creation screen.

Extensions:

- 3a. Email already exists: The system notifies the user and suggests the "Forgot Password" flow.
- 3b. Invalid email format: The system prevents submission and highlights the error.

Special Requirements: Registration data must be transmitted over an encrypted connection.

Technology and data variations: Verification can be handled via a unique URL or a 6-digit TOTP code.

UC-T2: Registered User

Primary Actor:

- User: Wants secure, persistent access so they don't have to log in repeatedly.
- Developer: Needs to implement secure authentication to prevent unauthorized account access.

Stakeholders and interests: The user has a verified and active account.

Preconditions: The user has a verified and active account.

Success guarantee: The user is authenticated and a secure session is established.

Main success scenario:

1. User enters their registered email and password on the login screen.
2. The system hashes the input and compares it against the stored credentials.
3. The system generates a secure session token.
4. The system redirects the user to their "My Items" dashboard.
5. User selects "Logout," and the system invalidates the session token immediately

Extensions:

- 2a. Incorrect password: The system increments a "failed attempt" counter and prompts the user to try again.
- 2b. Account locked: After five failed attempts, the system temporarily disables the account for 15 minutes for security.

Special Requirements: Session tokens must be stored securely (e.g., HttpOnly cookies) to prevent XSS attacks.

Technology and data variations: Support for biometric login (FaceID/Fingerprint) on compatible mobile devices.

UC-T3: Profile Creation and Editing

Primary Actor: Registered User

Stakeholders and interests:

- User: Wants to provide contact details (e.g., phone number, secondary email) so finders can reach them.
- Finder: Needs accurate contact info to coordinate the return of a lost item.

Preconditions: User is logged into the application

Success guarantee: Profile information is updated and reflected across the user's active posts.

Main success scenario:

1. User navigates to the "Settings" or "Profile" tab.
2. User enters optional information, such as a profile picture or preferred contact method.
3. User clicks "Save Changes."
4. The system validates the data (e.g., phone number format) and updates the database.
5. The system displays a confirmation message.

Extensions:

- 4a. Invalid phone format: The system alerts the user and refuses the update until corrected.

Special Requirements: Users must have the option to hide their phone number from public view, showing it only after a claim is accepted.

Technology and data variations: Profile pictures can be uploaded from the local gallery or taken via the mobile camera.

UC-T4: Manage "My Items" Dashboard

Primary Actor: Registered User

Stakeholders and interests:

- User: Needs a centralized hub to monitor the status of every item they have reported as lost or found.
- System: Must ensure that changes made here (like marking an item as "found") sync across the entire platform.

Preconditions: User is logged in and has created at least one report.

Success guarantee: User can successfully view, edit, or resolve their own posts.

Main success scenario:

1. User selects the "My items" tab from the main navigation menu.
2. The system retrieves all active and past posts associated with that user's ID.
3. User views the status of each item (e.g., "Active," "Pending Claim," or "Resolved").
4. User selects a specific post to update its details or change its status.
5. The system saves the changes and updates the public feed accordingly.

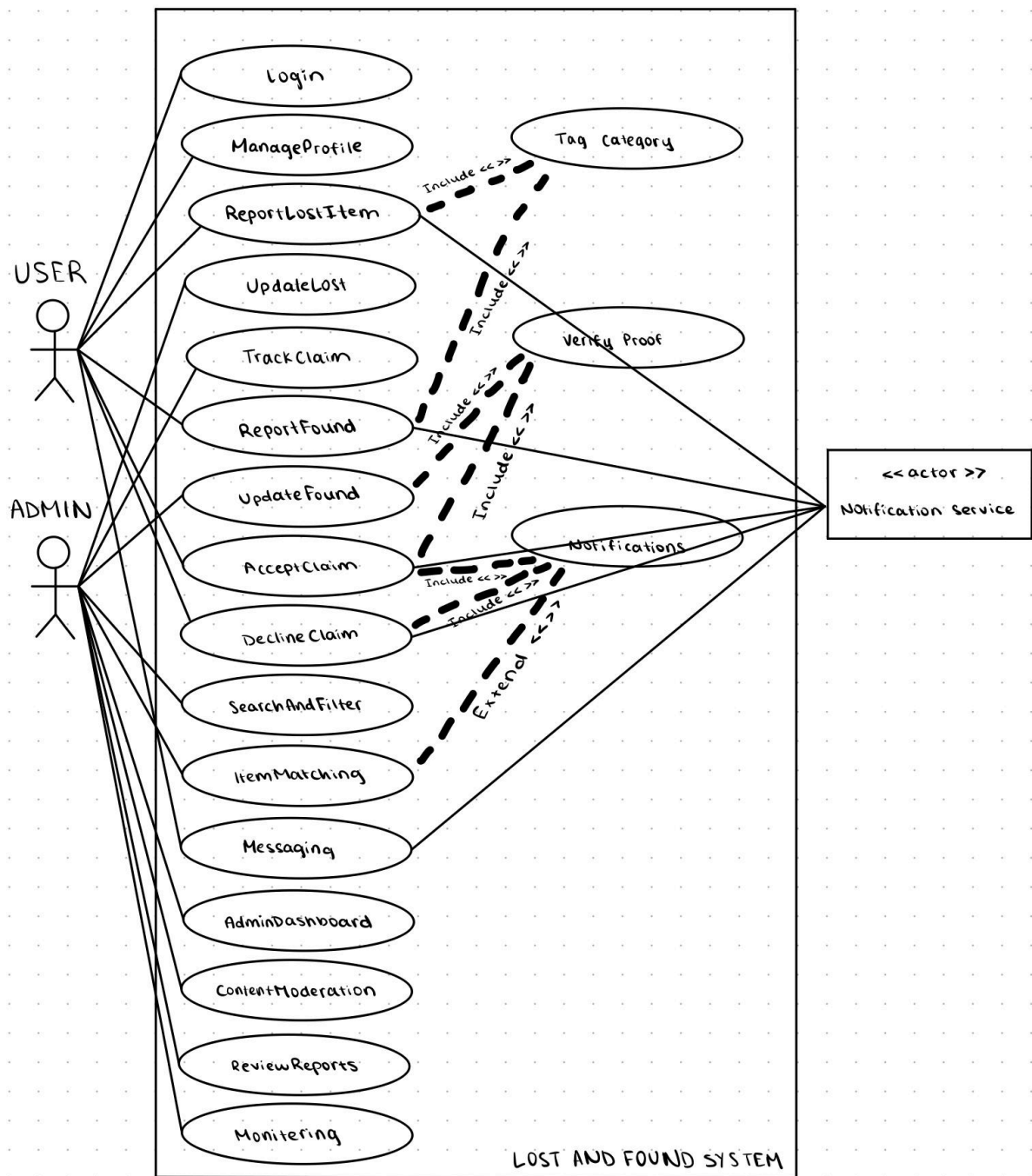
Extensions:

- 3a. No items found: The system displays a "No posts yet" message with a shortcut to report a new item.

Special Requirements: The dashboard must reflect real-time updates if a finder submits a claim request for the user's lost item.

Technology and data variations: Items can be filtered by "Lost" vs. "Found" or sorted by "Date Posted."

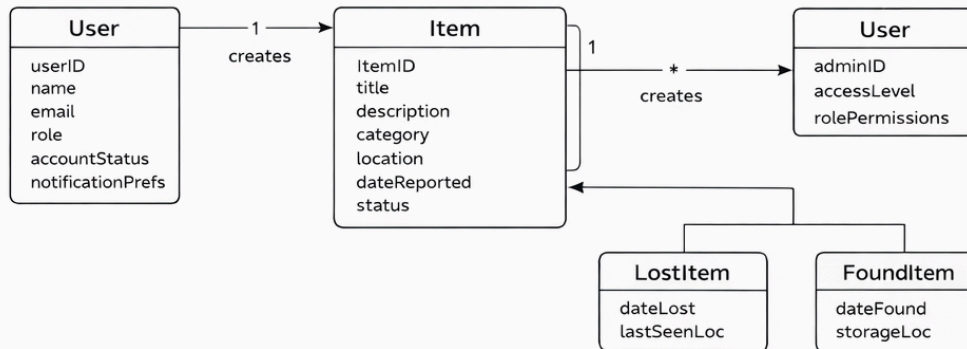
Use Case Diagram(s):



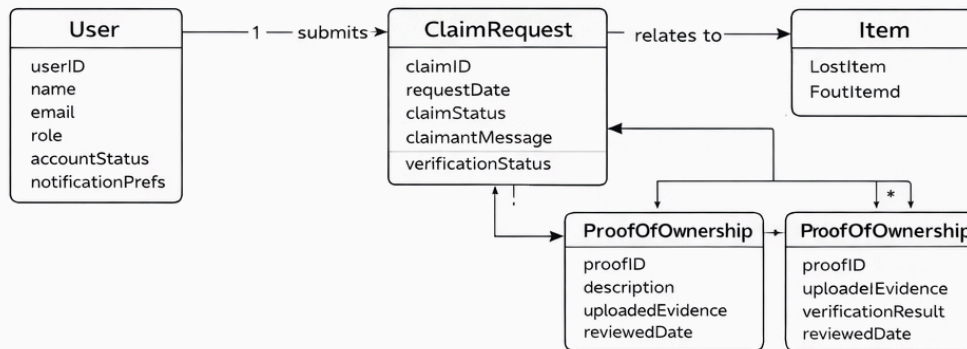
Conceptual Class Diagram(s):

Conceptual Class Diagrams

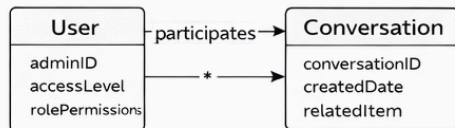
Conceptual Class Diagram 1 – Core Item Management Domain



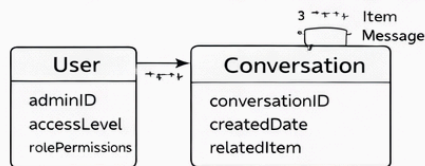
Conceptual Class Diagram 2 – Claims & Verification



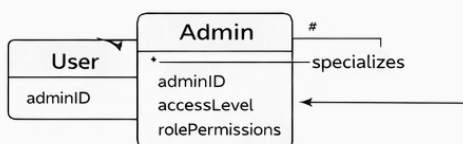
Conceptual Class Diagram 4 – Messaging & Notifications



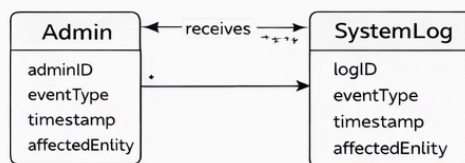
Conceptual Class Diagram 4 – Messaging &



Conceptual Class Diagram 5 –



Conceptual Class Diagram 5 – Administration & Moderation



Supplementary Specifications:

DJ — Secure & encrypted data transmission

This requirement ensures that the Campus Lost & Found App maintains a high standard of security by protecting all data in transit between the mobile client and the backend using Transport Layer Security (TLS) 1.3 to prevent "Man-in-the-Middle" (MITM) attacks. By enforcing HTTPS and implementing SSL Certificate Pinning, the system ensures that the app only communicates with authorized group servers, while the authentication backend utilizes JSON Web Tokens (JWT)—stored in secure, encrypted enclaves like the Android Keystore—to manage user states and prevent unauthorized session extraction. To further protect user privacy, passwords will be hashed using strong algorithms like bcrypt, and sensitive database fields will be shielded with AES-256-GCM encryption and strict API rate limiting to defend against brute-force attempts and SQL injection.

Ally — Real-time data consistency

Reliable real-time data consistency means the system must keep every user's view of a lost item accurate, up-to-date, and synchronized instantly whenever a change occurs. When a lost item's status is updated, such as being marked as "Found", that update must propagate across all devices and user sessions without delay. This prevents duplicate claims, eliminates confusion about whether an item is still missing, and ensures that search results, notifications, and claim workflows always reflect the true current state of the item. It supports trust in the platform by guaranteeing that everyone interacts with the same version of the data at all times.

Malavika — Fast & responsive UI

This requirement ensures that the system's interface responds quickly to user actions, such as reporting a found item, updating item status, or reviewing claim requests. The system should load pages, submit forms, and display updates without noticeable delays so users can complete tasks efficiently. This matters because slow or laggy interfaces can frustrate users and discourage them from using the platform to report or claim items. To support this requirement, the system must use efficient front-end design, optimized database queries, and responsive feedback (such as loading indicators or confirmation messages) so users experience smooth interactions throughout the application.

Unnati — Scalable search & notifications

Scalable search and notification performance means the system must stay fast and reliable even when many users are using the platform at the same time. When a user searches for items using common filters such as category, location, or status, the results should appear quickly without long loading times. This helps users easily find relevant lost or found posts without frustration. The notification system must also work efficiently as the number of users and posts grows. When a new item is posted, the system should quickly check for possible matches and notify the appropriate users without noticeable delay. This requirement is important because the platform may be used by many people across campus, especially during busy times. Efficient search processing and notification handling help ensure the system remains responsive and useful for everyone.

Taeoh — System reliability & admin access control

System reliability is fundamental to ensuring the Campus Lost & Found platform remains functional and accessible around the clock, aiming for a 99.5% uptime through automated daily backups and a swift recovery protocol that ensures users can report or search for items without interruption. In conjunction with this, secure admin access control is implemented through a Role-Based Access Control system, which restricts sensitive tasks like content moderation and user suspension to verified personnel while maintaining detailed audit logs of all administrative actions to guarantee both transparency and the integrity of user data.

GitHub Link: <https://github.com/malavika-cmd/CampusLostAndFoundApp.git>